Universidad Autónoma de Ciudad Juárez

Instituto de Ingeniería y Tecnología

Departamento de Ingeniería Eléctrica y Computación

Ingeniería de Software

"Sistema de Control de Gastos en Base al Modelo 4P+T: Mi-Monederito"

Equipo THETO

Sariña Salinas Luis Eduardo	**202245**
Esquivel García Ian Gerardo	**234646**
Beltrán Herrada Jared Gustavo	**234878**
Santillan Núñez Ana Paula	**246102**
Muñoz González Marlene Itzel	**247554**

Fundamentos de Ingeniería de Software

Maestro. Rafael Jiménez Castro

Ciudad Juárez, Chihuahua, Enero - Mayo 2026

4. Personas

4.1. Roles

En esta etapa tomamos la base de nuestra metodología considerando que presenta tres roles principales

4.1.1. Product Owner

Es la persona responsable de definir qué debe desarrollarse en el producto y de priorizar las tareas en el Product Backlog, es importante porque asegura que el equipo trabaje en las funciones que realmente aportan valor al cliente o al negocio.

Contemplando esas características principales identificamos que en nuestro equipo había una persona que podía desenvolverse confiadamente en este rol, por lo que decidimos darle la formalidad del rol a Jared Beltrán, además de que sabemos que el se puede desenvolver en este rol, también consideramos que su participación desde esta perspectiva ayudará al resto del grupo en el proyecto.

4.1.2. Scrum Master

Es el facilitador del equipo, se encarga de que el equipo siga correctamente la metodología Scrum y de eliminar obstáculos durante el Sprint, su función comprende ayudar al equipo a trabajar de forma eficiente y a mejorar continuamente el proceso.

Desde nuestro equipo tomamos la decisión de que Ana Santillán desempeñe este rol en la ejecución del proyecto, creemos que es un rol que exige habilidades que podemos encontrar en este integrante del equipo.

4.1.3. Development Team

Es el grupo de profesionales que diseña, desarrolla y prueba el producto durante cada sprint, son parte importante porque son quienes convierten las ideas y requisitos en un producto funcional.

Aunque en esta metodología no define otros roles para el equipo de desarrollo, particularmente nosotros definimos algunos, que nos ayudan a determinar cuales son las responsabilidades de cada uno.

4.1.3.1. Responsable de Procesos:

Es el miembro del equipo encargado de diseñar, organizar y supervisar la lógica de funcionamiento de los procesos internos del sistema, en nuestro proyecto desempeñado por Luis Sariñana

4.1.3.2. Responsable de Interfaz de Usuario:

Encargado del diseño y desarrollo de la interfaz visual del sistema, asegurando que la aplicación sea intuitiva, clara y fácil de usar para los usuarios, esta parte del proceso será desempeñada por Ian Esquivel.

4.1.3.3. Responsable de Base de Datos y Pruebas:

Encargado de la estructuración, administración y optimización de la base de datos del sistema, garantizando la correcta gestión y almacenamiento de la información, además es el miembro del equipo encargado de verificar que el sistema funcione correctamente y cumpla con los requisitos establecidos, esto implica realizar pruebas sobre las distintas funcionalidades del sistema para detectar errores, fallos o inconsistencias antes de que el producto sea entregado, rol desempeñado por Marlene Muñoz.

4.1.4. Resumen de roles

Roles		**Integrante**
Product Owner		Jared Beltran
Scrum Master		Ana Santillan
Development Team	Responsable de Procesos	Luis Sariñana
	Responsable de Interfaz de Usuario	Ian Esquivel
	Responsable de Bases de Datos y Pruebas	Marlene Muñoz

Tabla 1. Resumen de roles del equipo conforme a la metodología de Autoría propia

4.2. Responsabilidades

Roles		**Responsabilidades**
Product Owner		Definir la visión del producto, gestionar y priorizar el Product Backlog, comunicar requisitos al equipo, validar las funcionalidades desarrolladas y mantener comunicación con los interesados del proyecto.
Scrum Master		Facilitar las reuniones del equipo, eliminar obstáculos que afecten el desarrollo, guiar al equipo en las prácticas de Scrum, fomentar la comunicación y apoyar la mejora continua del proceso.
Development Team	*Responsable de Procesos*	Diseño de la lógica de negocio, validación de flujos de datos y estructuración de los procesos internos de cálculo.

	Responsable de Interfaz de Usuario	Diseño de las pantallas del sistema, implementación de componentes visuales y mejora de la experiencia de usuario en las funcionalidades principales de la plataforma.
	Responsable de Base de Datos y Pruebas	Diseño del esquema de la base de datos, gestión de consultas y mantenimiento de la integridad de los datos utilizados por el sistema, ejecución de pruebas funcionales del sistema, detección y reporte de errores, así como validación del correcto funcionamiento de las funcionalidades desarrolladas.

Tabla 2. Descripción de responsabilidades de cada rol de Autoría propia

4.3. Estrategia de comunicación

Para asegurar la agilidad que tiene la metodología SCRUM, el equipo utilizará un esquema de comunicación híbrido:

- *Gestión de tareas y avance:* Se utilizará Jira para dar seguimiento a tareas, cronogramas e indicadores de avance.
- *Comunicación técnica y código:* Se usará GitHub para la revisión de código y retroalimentación técnica.
- *Comunicación asincrónica*: Se llevarán a cabo reuniones periódicas (acordes a los ciclos establecidos en los Sprints) para la revisión de avances y coordinación.
- *Comunicación inmediata:* Se utilizará WhatsApp para la resolución de dudas rápidas y avisos urgentes que no requieran ser documentados.

4.4. Reglas internas del equipo

El equipo seguirá las siguientes reglas para asegurar la calidad y el cumplimiento de objetivos del proyecto:
Aislamiento de cambios: Todo nuevo desarrollo deberá trabajarse en una rama independiente para no comprometer la versión estable.
Validación cruzada: Ningún integrante del equipo puede subir código a la rama principal (versión estable) si no ha sido revisada y aprobada anteriormente por algún otro miembro del equipo, esto con el fin de detectar errores omitidos por el autor original.
Criterios de aceptación: Una tarea sólo será marcada como finalizada en "Jira" si cumple con los requisitos anteriormente definidos.
Prioridad de bugs: La corrección de bugs o errores tendrá una prioridad absoluta, esto con el fin de asegurar la mayor calidad del producto.

4.5. Reglas para el manejo de conflictos

Para resolver cualquier conflicto de índole técnico o de procesos, se seguirán los siguientes pasos:

- Referencia al product backlog: Ante dudas sobre que funcionalidad se debe desarrollar, siempre se tendrá como referencia la priorización establecida en Product Backlog.
- Consenso de retrospectivas: Se hará uso de sesiones de Retrospective al final de
- cada sprint, con el fin de detallar estragos en el flujo de trabajo.
- Escalamiento preventivo: Se mantendrá énfasis en el flujo de trabajo para identificar retrasos y centrarnos en las prioridades, esto con el fin de evitar conflictos de entrega.

5. Producto

5.1. Funcionalidades principales

- *Dashboard Mensual de control:*
  Pantalla principal que muestra de forma gráfica la distribución del dinero entre viviendo, Servicios de Escuela y Otros, permitiendo un balance inmediato entre ingresos y Egresos
- *Gestión de Guardaditos:*
  Función para separar montos de dinero que no pueden ser utilizados hasta el final del mes, ayudando a evitar el gasto impulsivo.
- *Módulo de planes de Ahorro:*
  Permite configurar metas específicas (ejemplo: $2000 en 10 meses) calculando automáticamente el resguardo mensual necesario.
- *Control de Préstamos:*
  Herramienta para organizar y dar seguimiento a los pagos de deudas o préstamos realizados, centralizando los compromisos financieros.
- *Límites de Gastos por categoría:*
  Permite establecer topes máximos de gasto (ejemplo: $200 para ocio) con indicadores visuales de proximidad al límite.
- *Historial detallado*:
  Listado cronológico de entradas y salidas para auditoría personal del usuario.

5.2. Requisitos Básicos

Para que el sistema funcione correctamente según lo planeado por equipo de desarrollo, se deben cumplir los siguientes requisitos mínimos:

5.2.1. Requisitos Funcionales (RF)

- *RF-01 Registro y Acceso:*

El sistema debe permitir la creación de cuentas (Nombre, Correo, Contraseña) y validación de identidad para proteger los datos financieros.

- *RF-02 Gestión de Transacciones:*
  El usuario debe poder sumar ingresos y restar egresos de forma manual, vinculándolos a una categoría.
- *RF-03 Calculo automático de Ahorros:*
  El sistema debe calcular la cuota mensual necesaria cuando el usuario ingresa un monto total y un plazo en meses.
- *RF-04 Visualización de Estadísticas*:
  El sistema debe transformar los datos numéricos en gráficos de distribución porcentual de forma automática.

5.2.2. Requisitos No Funcionales (RNF)

- *RNF-01 Persistencia de Datos:*
  La información debe almacenarse de forma estructurada para que el historial sea accesible en cualquier momento (Responsabilidad del área de Base de Datos).
- *RNF-02 Intuitividad de Interfaz:*
  El diseño debe seguir el esquema de navegación de 3 a 4 clics máximo para funciones críticas, manteniendo la estética presentada en el prototipo
- *RNF-03 Rendimiento:*
  El dashboard debe cargar y actualizar los saldos en un tiempo menor a 2 segundos tras realizar un registro.
- *RNF-04 Seguridad de Perfil:*
  Posibilidad de cerrar sesión (Logout) para evitar que terceros accedan a la información financiera en el mismo dispositivo.

5.3. Características del software

El sistema Mi-Monederito debe tener algunas características importantes, las cuales deben hacer que la aplicación sea confiable, fácil de usar y útil para los estudiantes.

5.3.1. Usabilidad

La usabilidad es muy importante en el sistema. Los estudiantes son los que van a usar la aplicación para registrar sus gastos. Por eso, el diseño de la aplicación debe hacer que sea fácil moverse por la aplicación sobre todo en:

- Registro de ingresos.
- Registro de gastos.
- Visualización de reportes.
- Metas de Ahorro.
- Alertas financieras.

También es importante que la aplicación minimice la curva de aprendizaje sin necesidad de usar instrucciones complejas.

5.3.2. Rendimiento y rapidez

El sistema debe ofrecer respuestas rápidas al usuario cuando este registre una transacción, consulte reportes o revise su saldo disponible.
Las operaciones principales del sistema incluyen:

- Guardar una transacción financiera.
- Actualizar el saldo del usuario.
- Calcular reportes semanales o mensuales.

Un buen rendimiento aumenta la probabilidad de que el sistema sea más utilizado.

5.3.3. Seguridad

Aunque el sistema no realiza transacciones bancarias reales, la información sigue siendo sensible. Por esta razón, el sistema debe implementar seguridad.
Entre los aspectos que se debe considerar la aplicación se encuentran:

- Protección de los datos personales del usuario.
- Prevención de pérdida de información.
- Manejo seguro del almacenamiento de datos.

5.3.4. Confiabilidad

El sistema debe garantizar que la información registrada sea almacenada correctamente y que los cálculos financieros sean precisos.
Por ejemplo, cuando un usuario registra un gasto, el sistema debe actualizar correctamente:

- El saldo actual.
- El balance de categorías.
- El progreso de metas de ahorro.

Si el sistema presenta errores en estos cálculos, el usuario podría perder confianza.

5.3.5. Disponibilidad

La aplicación debe estar disponible para los usuarios en todo momento, así pueden registrar gastos cuando lo necesiten.

5.3.6. Escalabilidad

El sistema debe diseñarse para que sea fácil agregar nuevas funciones más adelante. Por ejemplo, en versiones futuras podrían añadirse características como:

- Sincronización con servicios financieros.
- Exportación de reports.
- Análisis más avanzados de hábitos de gasto.
- Integración con servicios en la nube.

Por lo que diseñar el sistema con una arquitectura escalable facilitará los cambios sin necesidad de rediseñar todo.

5.4. Criterios de calidad

Para garantizar que el sistema Mi-Monederito cumpla con los objetivos del proyecto, se establecen criterios de calidad que permitan evaluar el funcionamiento.

5.4.1. Correcto funcionamiento de las funcionalidades

El sistema debe implementar correctamente las funciones principales definidas en el proyecto, entre ellas:

- Registro de ingresos.
- Registro de gastos.
- Clasificación por categorías.
- Reportes de consumo.
- Metas de Ahorro.
- Sistema de alertas.

5.4.2. Precisión de los cálculos

El sistema debe realizar correctamente los cálculos relacionados con:

- Balance financiero.
- Total de gastos por categoría.
- Progreso de metas de Ahorro.
- Análisis de consumo semanal o mensual.

La precisión de estos cálculos es para que los usuarios puedan confiar en la información que proporcionada.

5.4.3. Experiencia de usuario

Para evaluar esto se pueden realizar pruebas de usabilidad con estudiantes, verificando que puedan completar tareas como:

- Registrar un gasto.
- Consultar un reporte.
- Crear una meta de Ahorro.

Si los usuarios encuentran dificultades para realizar estas tareas, el diseño del sistema debe ser mejorado.

5.4.4. Mantenibilidad del sistema

El sistema debe ser desarrollado con código organizado y documentado para facilitar futuras modificaciones o mejoras.
El uso de herramientas como GitHub para llevar un control de las diferentes versiones y revisiones del código nos ayuda a mantener un orden. Además, GitHub nos permite trabajar juntos de forma colaborativa.

5.4.5. Pruebas del sistema

Antes de considerar que una funcionalidad está terminada, esta debe pasar diferentes tipos de pruebas:

- Pruebas funcionales para verificar que las funciones estén haciendo lo deseado.
- Pruebas de usabilidad para evaluar la interacción del usuario con Mi Monederito.
- Pruebas de rendimiento para verificar la rapidez del sistema.

Estas pruebas ayudan a detectar errores.

5.5. ¿Qué haría que el producto fracase?

El fracaso del sistema puede ocurrir si ciertos riesgos no son identificados o gestionados adecuadamente.

5.5.1. Falta de facilidad de uso

Si una aplicación es complicada o lleva mucho tiempo para registrar un gasto, los usuarios no la utilizarán. Una mala experiencia de usuario puede hacer que una aplicación fracase.

5.5.2. Problemas técnicos o fallos del sistema

Errores en el código, fallos en la base de datos o problemas de rendimiento pueden hacer que la aplicación deje de funcionar. Estos problemas afectan la confiabilidad de la aplicación.

5.5.3. Falta de mantenimiento o mejoras

Una aplicación que no recibe actualizaciones ni mejoras con el tiempo puede volverse obsoleta. Así que, es importante mantener el sistema actualizado para adaptarse a las necesidades de los usuarios.

5.5.4. Falta de valor para el usuario

Si los estudiantes no sienten que la aplicación les ayuda realmente a administrar su dinero, es posible que la abandonen. Por esta razón, nos debemos enfocar en resolver de forma efectiva el problema.

6. Tecnología

6.1. Lenguaje elegido

Vamos a usar el framework de Ionic, el cual está basado en HTML, CSS y TypeScript. Esto permite utilizar, si se desea, otros frameworks de JavaScript/TypeScript como Angular, React o Vue. Servirá para crear la aplicación como si fuera una página web, lo

que facilitará el desarrollo de una forma más fácil comparado con otras alternativas de desarrollo móvil nativo. Además, Ionic permite crear aplicaciones multiplataforma, lo que significa que una misma base de código puede utilizarse para generar aplicaciones que funcionen tanto en dispositivos Android como en navegadores web. Otra ventaja de utilizar Ionic es que cuenta con una gran cantidad de componentes visuales ya preparados, como botones, menús, formularios y barras de navegación, lo que permite desarrollar interfaces de usuario de forma más rápida. También cuenta con una comunidad amplia y buena documentación, lo que facilita resolver problemas durante el desarrollo del proyecto.

6.2. Herramientas

Las herramientas que se utilizarán durante el desarrollo del proyecto son las siguientes:

- Ionic Framework
- Visual Studio Code
- GitHub

Ionic Framework será utilizado como la base principal para la creación de la aplicación. Visual Studio Code se utilizara como editor de código debido a su facilidad de uso, compatibilidad con múltiples lenguajes y extensiones que ayudan al desarrollo. GitHub permitirá almacenar el código del proyecto en línea y facilitar el trabajo colaborativo entre los integrantes del equipo.

6.3. Control de versiones (en GitHub)

Utilizaremos GitHub para poder controlar qué hacemos y qué cambios se van realizando en el proyecto. Cada modificación realizada en el código puede registrarse mediante commits, los cuales permiten dejar mensajes descriptivos como por ejemplo ”Fix de xxxx.o ”Función xxxx implementada”. Esto permite llevar un historial de cambios del proyecto y facilita regresar a versiones anteriores en caso de que algo deje de funcionar correctamente. Además, el control de versiones ayuda a que varias personas puedan trabajar en el mismo proyecto sin sobrescribir el trabajo de otros. La versión enviada se puede visualizar según la gráfica mostrada abajo, la cual representa el historial de cambios del repositorio.

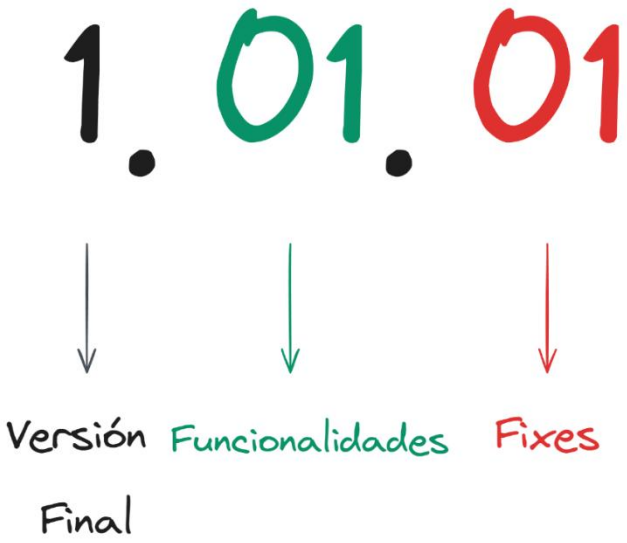

Figura 1. Descripción de versiones de Autoría propia

6.4. Herramientas de prueba

Esta parte puede actualizarse a futuro mientras vamos descubriendo cómo se utilizan herramientas específicas para pruebas automatizadas. Algunas opciones que podrían utilizarse son Jasmine o Karma, las cuales permiten realizar pruebas de manera automática sobre el código desarrollado. Sin embargo, durante las primeras etapas del proyecto es probable que se realicen principalmente pruebas manuales. Estas pruebas pueden llevarse a cabo ejecutando la aplicación en navegadores o en dispositivos móviles reales. También se puede pedir apoyo a personas externas, como familiares o amigos, para probar la aplicación y obtener retroalimentación sobre su funcionamiento y facilidad de uso.

6.5. Infraestructura

La infraestructura utilizada para el desarrollo del proyecto incluye los siguientes elementos:

- El desarrollo se realizará en Visual Studio Code utilizando Node.js para gestionar dependencias y ejecutar el entorno de desarrollo.
- El código del proyecto se almacenará en repositorios de GitHub.
- El servidor de pruebas podrá ejecutarse localmente durante el desarrollo utilizando las herramientas de Ionic.
- El servidor final estará alojado en la nube para permitir el acceso a la aplicación desde diferentes dispositivos.
- Para la compilación de la aplicación en dispositivos Android se podrá utilizar Android Studio.

7. Referencias

[1] Salimi, S. (s.f.). Fail fast. Agile Academy. Disponible en: https://www. agile-academy.com/es/diccionario-agil/fallar-rapido/
[2] Villa, A. A. (2025, September 1). Qué es Smoke Test. Profile Software Services. Disponible en: https://profile.es/blog/que-es-smoke-test/

8. Anexos

8.1. Bitácora

Como tal no es la definición de una bitácora, solo será una sección en el documento donde semanalmente subiremos un reporte de nuestros avances a través de evidencias en jira, como esta sección tiene relación con la forma de evaluación y no directamente con el proyecto es posible que esta sección sea eliminada en la entrega final o se preserve para efectos de documentación.

Sprint 1.

- Semana 1 (01 - 08 de marzo 2026)
  En esta semana teníamos como meta las primeras tres secciones de la documentación, por lo que algunas secciones se dividieron en subactividades, pero finalmente terminamos las 3 con éxito, luego de la revisión estuvimos de acuerdo con la entrega del documento y procedimos, a definir los planes de trabajo de la siguiente semana.

Actividad	Persona asignada	Prioridad	Estado	Start date	Fecha
~~SGGA-1~~ Definir el Proyecto	Sin asignar	Highest	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-2~~ Creación de espacio en Jira	ANA PAULA SANTILL...	Medium	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-4~~ Definición del Product Backlog	ANA PAULA SANTILL...	Medium	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-18~~ Documentacion: Seccion 1	JARED GUSTAVO BEL...	Low	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-19~~ Documentacion:Seccion 2	luis Eduardo Sariñana...	Low	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-20~~ Documentacion:Seccion 3	ANA PAULA SANTILL...	Low	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-31~~ Planeacion del Proyecto	ANA PAULA SANTILL...	Medium	FINALIZADA	01 mar 2026	08 mar 2026
~~SGGA-5~~ Análisis de requerimientos del sistema	JARED GUSTAVO BEL...	Highest	EN CURSO	08 mar 2026	15 mar 2026

Figura 2. Fragmento Backlog semana del 01 al 08 de marzo de Autoría propia

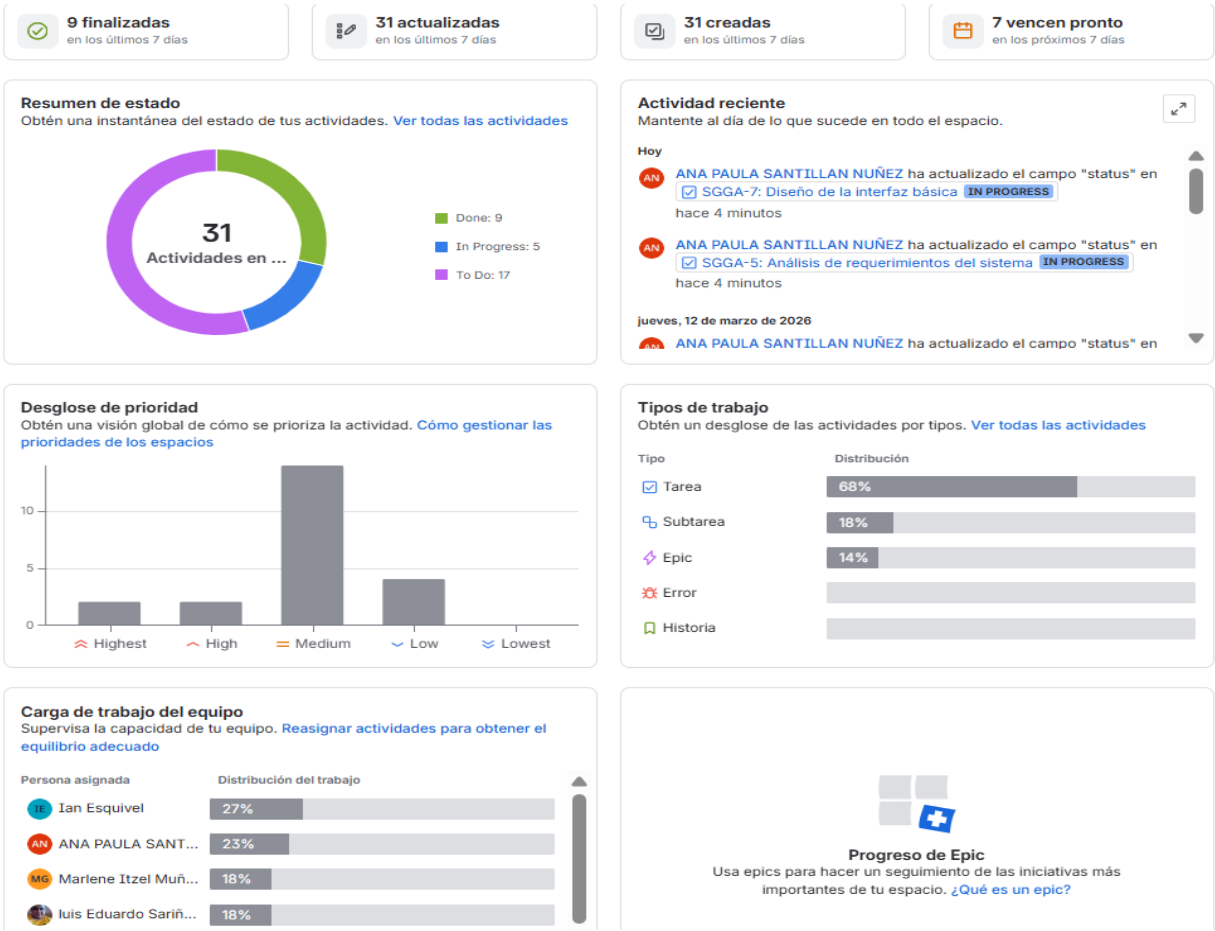

Figura 3. Fragmento del dashboard de Jira de Autoría propia.

- Semana 2 (09 - 15 de marzo 2026)

Nuevamente esta semana teníamos como meta las otras tres secciones de la documentación, por lo que algunas secciones se dividieron en subactividades, pero finalmente terminamos las 3 con éxito, luego de la revisión estuvimos de acuerdo con la entrega del documento y procedimos, a definir los planes de trabajo de la siguiente semana, además realizamos otras decisiones en conjunto y discutimos sobre la estructura del proyecto pues teníamos que definir algunas parte como equipo para que todos estuviéramos acorde, por tanto hicimos un pequeño diseño de la aplicación.

Figura 4. Diseño informal del proyecto de Autoría propia

En las siguientes figuras se observa el avance final de esta semana, hicimos una pausa en las actividades SGGA-6 y SGGA-7 porque recibimos la observación por parte de nuestro docente, aclarando que en esta etapa no era necesaria la codificación del proyecto.

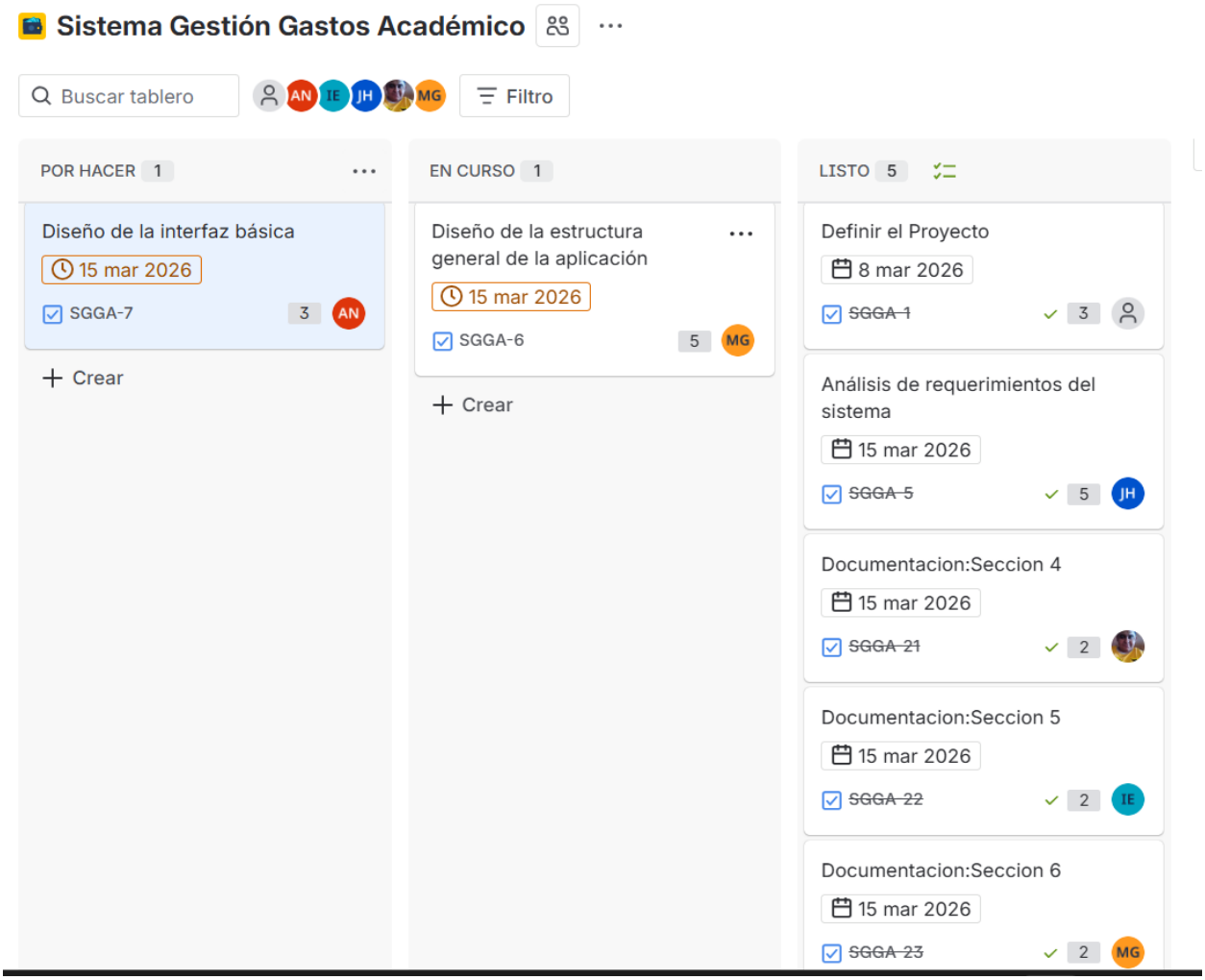

Figura 5. Kanban del proyecto semana 08 al 15 de marzo de Autoría propia

A continuación mostramos el estatus del sprint 1 al llegar a su fin.

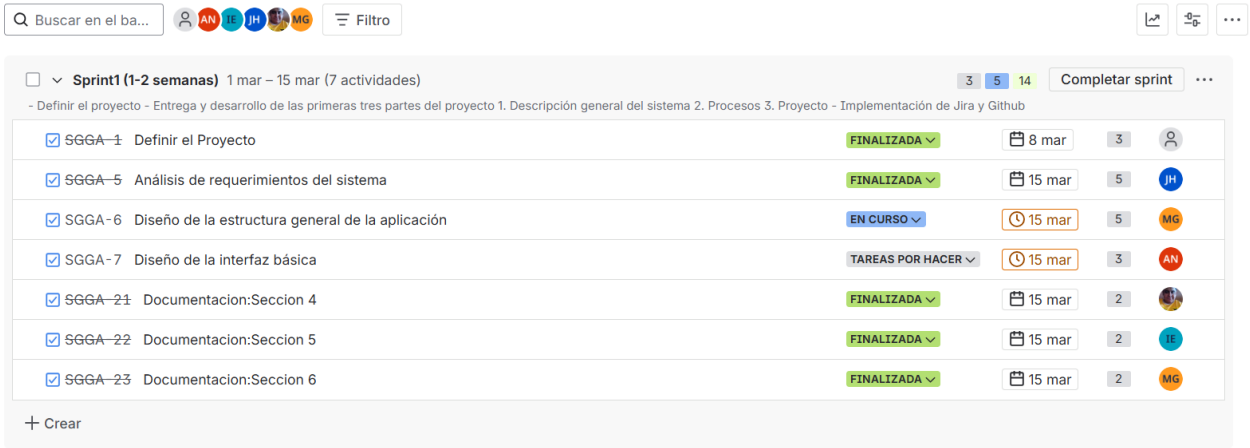

Figura 6. Sprint 1 semana 08 al 15 de marzo de Autoría propia

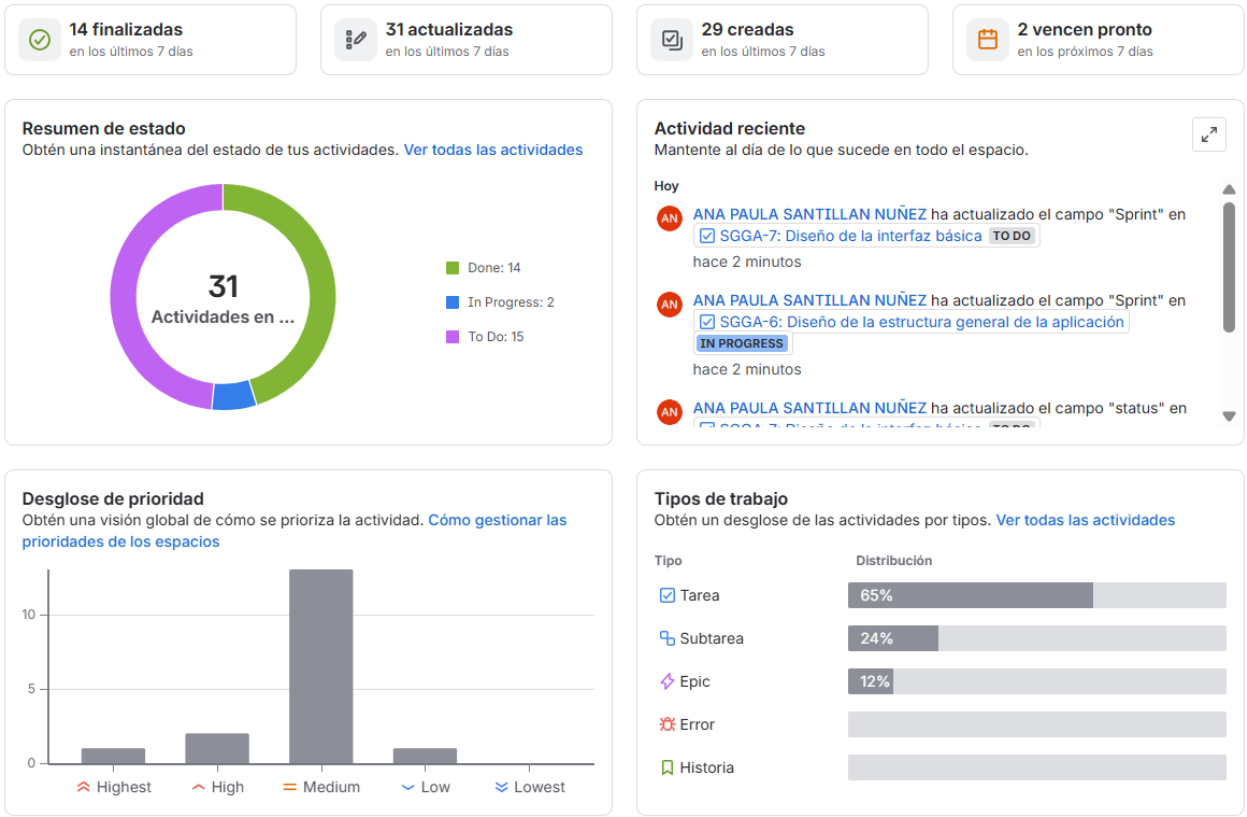

Figura 7. Dashboard de la semana 08 al 15 de marzo de Autoría propia

8.2. Repositorio de Github

Usamos GitHub en este proyecto porque nos permite gestionar y colaborar en el código de manera organizada y segura. A través de su sistema de control de versiones basado en Git, podemos registrar todos los cambios realizados en el proyecto, facilitar que varios desarrolladores trabajen al mismo tiempo sin perder historial y revisar modificaciones antes de integrarlas al código principal. Además, GitHub ofrece herramientas para seguimiento de tareas, revisión de código y automatización de procesos, lo que mejora la calidad del software y hace más eficiente el trabajo en equipo durante el desarrollo del proyecto.

[Repositorio de Github Sistema Control Gastos (bajo el modelo 4P+T)](https://github.com/AnaSantillanal246102/Sistema_Control_Gastos_4PT)

Link del Repositorio: https://github.com/AnaSantillanal246102/Sistema_Control_Gastos_4PT

9. Tablas

9.1. Tabla de contenido

1. Descripción general del sistema. ..... **Error! Bookmark not defined.**
   1.1. Nombre..... **Error! Bookmark not defined.**
   1.2. Problema que resuelve..... **Error! Bookmark not defined.**
   1.3. Usuarios principales ..... **Error! Bookmark not defined.**
   1.4. Objetivo general ..... **Error! Bookmark not defined.**
   1.5 Objetivos Específicos ..... **Error! Bookmark not defined.**
2. Procesos (de la aplicación) ..... **Error! Bookmark not defined.**
   2.1 Perfil de usuarios..... **Error! Bookmark not defined.**
   2.2 Casos de Uso Principales..... **Error! Bookmark not defined.**
   2.3 Descripción de los Procesos Principales ..... **Error! Bookmark not defined.**
      2.3.1. Proceso 1: Registro y categorización Inteligente..... **Error! Bookmark not defined.**
      2.3.2. Proceso 2: Monitoreo de Alertas de Solvencia ..... **Error! Bookmark not defined.**
   2.4. Proceso de desarrollo..... **Error! Bookmark not defined.**
      2.4.1. Metodología de desarrollo elegida ..... **Error! Bookmark not defined.**
      2.4.2. Justificación ..... **Error! Bookmark not defined.**
   2.5. Actividades principales ..... **Error! Bookmark not defined.**
      2.5.1. Análisis / Product Backlog..... **Error! Bookmark not defined.**
      2.5.2. Diseño / Planificación..... **Error! Bookmark not defined.**
      2.5.3. Desarrollo..... **Error! Bookmark not defined.**
      2.5.4.Pruebas ..... **Error! Bookmark not defined.**
      2.5.5. Entrega / Review / Retrospective ..... **Error! Bookmark not defined.**
   2.6. Flujo de trabajo..... **Error! Bookmark not defined.**
   2.7. Control de calidad..... **Error! Bookmark not defined.**
      2.7.1. ¿Cómo evitarán errores? ..... **Error! Bookmark not defined.**
      2.7.2. ¿Qué prácticas utilizaran?..... **Error! Bookmark not defined.**
3. Proyecto ..... **Error! Bookmark not defined.**
   3.1. Definición del proyecto ..... **Error! Bookmark not defined.**
      3.1.1. Alcance ..... **Error! Bookmark not defined.**

3.1.2. Entregables ..... **Error! Bookmark not defined.**
3.2. Planificación básica (en Jira) ..... **Error! Bookmark not defined.**
3.2.1. Cronograma ..... **Error! Bookmark not defined.**
3.2.2. Lista de Actividades - Backlog ..... **Error! Bookmark not defined.**
3.2.3. Sprints ..... **Error! Bookmark not defined.**
3.3. Recursos ..... **Error! Bookmark not defined.**
3.3.1. Humanos ..... **Error! Bookmark not defined.**
3.3.2. Técnicos ..... **Error! Bookmark not defined.**
3.4. Riesgos identificados ..... **Error! Bookmark not defined.**
3.5. Métricas del proyecto ..... **Error! Bookmark not defined.**
3.5.1. Tiempo ..... **Error! Bookmark not defined.**
3.5.2. Costo estimado ..... **Error! Bookmark not defined.**
3.5.3. Indicadores del avance ..... **Error! Bookmark not defined.**
4. Personas 1
4.1. Roles 2
4.1.1. Product Owner 2
4.1.2. Scrum Master 2
4.1.3. Development Team 2
4.1.4. Resumen de roles 3
4.2. Responsabilidades 3
4.3. Estrategia de comunicación 4
4.4. Reglas internas del equipo 4
4.5. Reglas para el manejo de conflictos 5
5. Producto 5
5.1. Funcionalidades principales 5
5.2. Requisitos Básicos 5
5.2.1. Requisitos Funcionales (RF) 5
5.2.2. Requisitos No Funcionales (RNF) 6
5.3. Características del software 6
5.3.1. Usabilidad 6
5.3.2. Rendimiento y rapidez 7
5.3.3. Seguridad 7
5.3.4. Confiabilidad 7

5.3.5. Disponibilidad 7
5.3.6. Escalabilidad 7
5.4. Criterios de calidad 8
5.4.1. Correcto funcionamiento de las funcionalidades 8
5.4.2. Precisión de los cálculos 8
5.4.3. Experiencia de usuario 8
5.4.4. Mantenibilidad del sistema 8
5.4.5. Pruebas del sistema 9
5.5. ¿Qué haría que el producto fracase? 9
5.5.1. Falta de facilidad de uso 9
5.5.2. Problemas técnicos o fallos del sistema 9
5.5.3. Falta de mantenimiento o mejoras 9
5.5.4. Falta de valor para el usuario 9
6. Tecnología 9
6.1. Lenguaje elegido 9
6.2. Herramientas 10
6.3. Control de versiones (en GitHub) 10
6.4. Herramientas de prueba 11
6.5. Infraestructura 11
7. Referencias 12
8. Anexos 12
8.1. Bitácora 12
8.2. Repositorio de Github 16
9. Tablas 17
9.1. Tabla de contenido 17
9.2. Índice de tablas 19
9.3. Tabla de figuras 20

9.2. Índice de tablas

Tabla 1. Registro detallado de los posibles riesgos del proyecto de Autoría Propia ..... **Error! Bookmark not defined.**
Tabla 2. Resumen de roles del equipo conforme a la metodología de Autoría propia 3
Tabla 3. Descripción de responsabilidades de cada rol de Autoría propia 4

9.3. Tabla de figuras

Figura 1. Diagrama de metodología de Autoría propia ..... **Error! Bookmark not defined.**
Figura 2. Cronograma de Proyecto Fragmento de planificación en Jira de Autoría propia ..**Error! Bookmark not defined.**
Figura 3. Fragmento del Backlog del proyecto en Jira de Autoría propia.... **Error! Bookmark not defined.**
Figura 4. Fragmento de Backlog del proyecto en Jira de Autoría propia .... **Error! Bookmark not defined.**
Figura 5. Imagen Sprint 1 del Proyecto en Jira de Autoría propia **Error! Bookmark not defined.**
Figura 6. Imagen Sprint 2 del Proyecto en Jira de Autoría propia**Error! Bookmark not defined.**
Figura 7. Imagen Sprint 3 del Proyecto en Jira de Autoría propia **Error! Bookmark not defined.**
Figura 8. Descripción de versiones de Autoría propia 11
Figura 9. Fragmento Backlog semana del 01 al 08 de marzo de Autoría propia.....12
Figura 10. Fragmento del dashboard de Jira de Autoría propia.....13
Figura 11. Diseño informal del proyecto de Autoría propia 14
Figura 12. Kanban del proyecto semana 08 al 15 de marzo de Autoría propia 15
Figura 13. Sprint 1 semana 08 al 15 de marzo de Autoría propia.....15
Figura 14. Dashboard de la semana 08 al 15 de marzo de Autoría propia 16